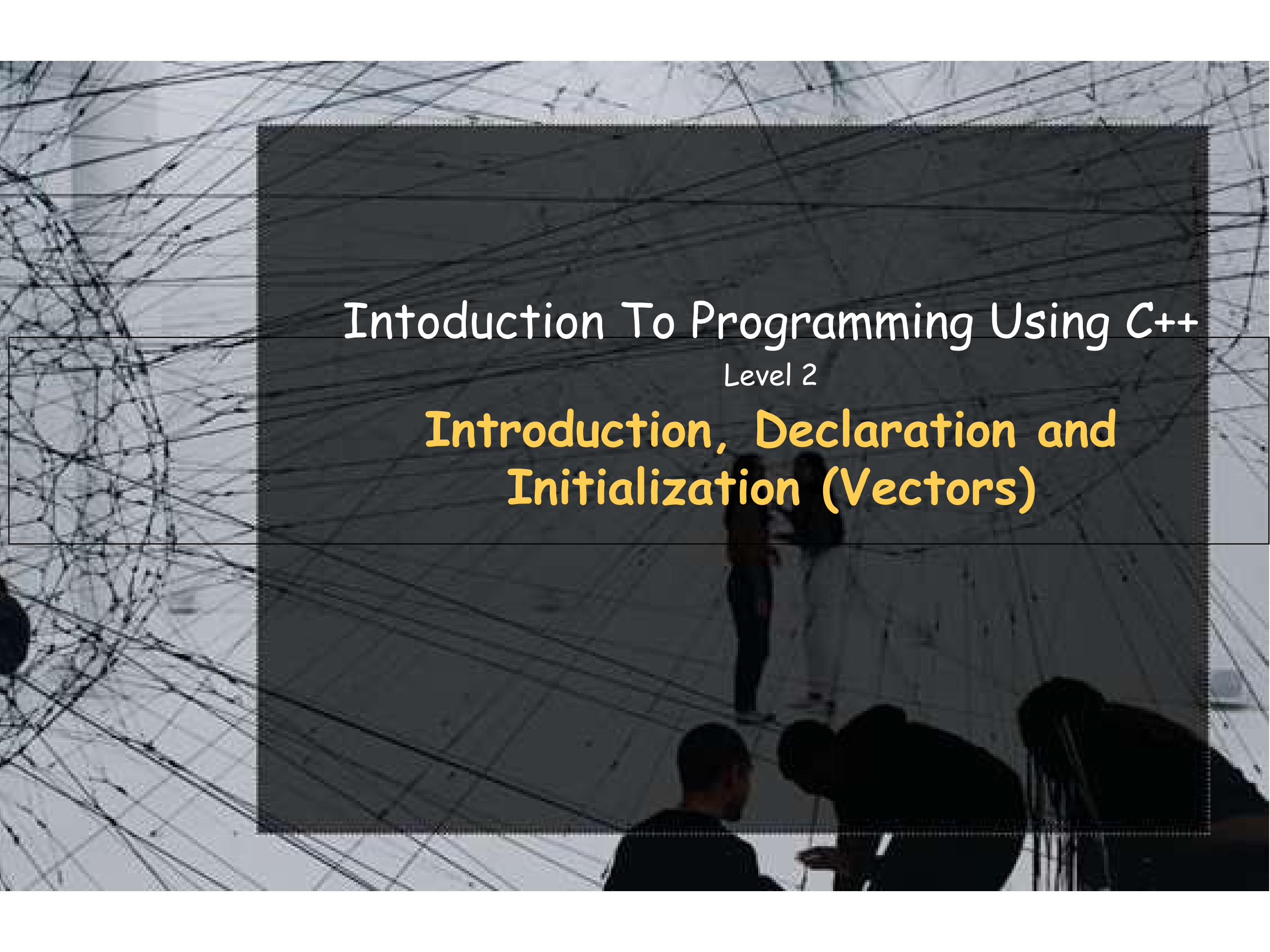




Introduction To Programming Using C++ Level 2

Introduction, Declaration and Initialization (Vectors)

The background of the slide is a grayscale photograph of a lecture hall. In the foreground, the silhouettes of several students are visible, some sitting and some standing, looking towards the front. In the background, a lecturer is standing near a large screen or whiteboard. The overall scene is dimly lit, with the main light source coming from the front, creating a silhouette effect.

Introduction To Programming Using C++

Level 2

Introduction, Declaration and Initialization (Vectors)

Definition

- ◆ A `Vector` is a dynamic array provided by the C++ Standard Library.
- ◆ Unlike a regular Array, a Vector can automatically grow or shrink when elements are added or removed.
- ◆ A Vector is useful when the number of elements is not known in advance.

Explanation

Regular Arrays vs Dynamic Data

-  In a regular Array, the size is fixed when the Array is defined.

-  We cannot change the number of elements of a built-in Array after its definition.

-  Sometimes, however, we do not know the required size at Compile Time.

-  C++ provides several ways to handle dynamic data. One traditional approach is Dynamic Arrays using Pointers, but manually managing them can be more complicated.

-  `Vector` provides a safer and easier way to work with a dynamically sized sequence of elements.

Declaration

```
vector<type> VectorName;
```

💡 Example

```
int main()
{
    vector<int> vNumbers = {10, 20, 30, 40};

    for (int number : vNumbers)
    {
        cout << number << " ";
    }
    return 0;
}
```

➡ Result

10 20 30 40

✚ Note

- ◆ For small data types such as `int`, copying an element is usually inexpensive.
- ◆ For large objects such as large `struct` or `class` objects, repeated copying can have a greater performance and memory cost.
- ◆ If you only want to read the elements without modifying them, a `const` reference is often preferable:

```
for (const int &number : vNumbers)
{
    cout << number << " ";
}
```
- ◆ `const` prevents the loop from modifying the original elements.

⊘ Warning

- ◆ Be careful when using a non-`const` Reference:
for (int &number : vNumbers)
- ◆ Any modification made to `number` modifies the original element in the Vector.
- ◆ Use a Reference when you intentionally want to work with the original elements or avoid copying large objects.



Comparison

▨ Regular Array

- ✓ Fixed size.
- ✓ Size is determined when the Array is defined.
- ✓ The number of elements cannot be changed after definition.
- ✓ Example: ``int numbers[5];``

▤ Vector

- ✓ Dynamic size.
- ✓ Can grow or shrink as elements are added or removed.
- ✓ Does not require the size to be specified when declared.
- ✓ Example: ``vector<int> numbers;``

✦✦ Summary

- ✓ A regular Array has a fixed size, while a `Vector` has a dynamic size.
- ✓ A Vector can automatically grow or shrink as elements are added or removed.
- ✓ Use `vector<type> VectorName;` to declare a Vector.
- ✓ Use `push\_back()` to add an element to the end of a Vector.
- ✓ A Ranged `for` Loop can be used to iterate through Vector elements.
- ✓ `for (int number : vNumbers)` creates a copy of each element.
- ✓ `for (int &number : vNumbers)` uses a Reference to the original element.
- ✓ Use a Reference when you want to avoid unnecessary copying or intentionally modify the original elements.
- ✓ Use `const` Reference when you want to avoid copying while preventing modification.

Thank you 😊